

Comprehensive Project Documentation:

Cosmo Enterprises ERP System

Project Name: Cosmo Enterprises Management System

Project Type: B2B Enterprise Resource Planning (ERP) Web Application

Domain: Textile & Chemical Manufacturing

Development Framework: Python (Django)

1. Project Overview

Cosmo Enterprises is a comprehensive, enterprise-grade B2B (Business-to-Business) ERP web application designed specifically for a textile and chemical manufacturing company.

The system digitizes the entire workflow of the company, from managing client data and tracking manufacturing orders to handling secure online payments via Stripe, managing raw material inventory, and automatically sending professional email receipts and PDF invoices to clients.

Core Objectives

- **Centralized Management:** A single dashboard to manage clients, orders, batches, and finance.
 - **Secure Payments:** Integration with Stripe for secure, online payment links.
 - **Automation:** Automated email receipts and PDF invoice generation upon payment.
 - **Inventory Control:** Tracking raw materials and chemicals with low-stock alerts.
 - **Modern UI:** A highly professional, responsive, custom-built "Dark Mode" glassmorphism user interface.
-

2. Technology Stack

- **Backend Framework:** Python & Django (Robust, secure, and scalable).
 - **Frontend:** HTML5, CSS3 (Bootstrap 5 + Custom CSS Variables for Dark Theme), JavaScript (Vanilla JS for DOM manipulation).
 - **Database:** SQLite (Development).
 - **Payment Gateway:** Stripe API (RESTful API for Payment Links and Webhooks).
 - **Email Service:** Django [send_mail](#) via Gmail SMTP.
 - **PDF Generation:** WeasyPrint (Converts HTML templates to PDF).
 - **Version Control:** Git & GitHub.
-

3. System Architecture & Data Flow

The Admin Workflow (Internal)

1. **Login:** Admin authenticates via a secure, custom dark-themed login page.
2. **Client & Order Creation:** Admin adds a client and creates an order specifying the fabric type, quantity, and due date.
3. **Finance & Invoicing:** Admin generates an invoice for the order.
4. **Payment Generation:** Admin clicks "Generate Payment Link". Django talks to the Stripe API to create a secure checkout URL.
5. **Tracking:** Admin tracks the order status (Sample Preparing → Production → Quality Check → Ready for Shipment → Shipped).

The Client Workflow (External)

1. **Receiving Link:** The client receives the Stripe payment link via email or direct message from the admin.
2. **Checkout:** The client clicks the link, enters their card details on Stripe's secure, hosted page, and completes the payment.
3. **Confirmation:** The client sees a success page on Stripe.
4. **Automated Receipt:** Instantly, the Django backend receives the payment confirmation and automatically sends a beautifully formatted HTML email receipt to the client's inbox.

4. Database Schema (Backend Models)

The database is designed using Django's ORM (Object-Relational Mapper). Here are the core tables:

1. Client

Stores customer information.

- **Fields:** `name`, `email`, `phone`, `company`, `address`, `is_active`.
- **Relation:** One-to-Many with `Order`.

2. Order

Tracks the manufacturing orders.

- **Fields:** `order_id` (Auto-generated, e.g., ORD-0001), `fabric_type`, `quantity`, `payment` (Total Amount), `total_paid`, `due_date`, `status` (Choices: Sample Preparing, Production Starts, etc.).
- **Relation:** Foreign Key to `Client`. One-to-One with `Batch`.

3. Material (Inventory)

Tracks raw chemicals and fibers.

- **Fields:** `name`, `description`, `quantity` (Current Stock), `max_quantity`, `threshold` (Low stock alert level), `unit` (kg, liters), `reorder_level`.

4. Invoice

Tracks financial transactions.

- **Fields:** `invoice_id`, `amount`, `payment_method`, `stripe_payment_status` (pending/paid/failed), `stripe_payment_link`, `stripe_session_id`.
- **Relation:** Foreign Key to `Order`.

5. Batch

Tracks the manufacturing process of an order.

- **Fields:** `batch_number` (Auto-generated), `operator`, `start_time`, `end_time`, `status` (In Progress, Completed, Failed).
- **Relation:** One-to-One with `Order`.

6. ActivityLog

An audit trail for security and tracking.

- **Fields:** `activity_type`, `description`, `timestamp`, `user` (Admin who did it).

5. File Structure & Coding Logic

`mysite/` (Project Configuration)

- `settings.py`: The brain of the configuration. Contains database settings, installed apps, and crucial environment variables like `STRIPE_SECRET_KEY` and `EMAIL_HOST_PASSWORD`.
- `urls.py`: The root router that directs web traffic to the `core` app

`core/` (The Main Application)

- `models.py`: Defines the database tables (explained above).
- `views.py`: The most important file. It contains the Python functions that handle user requests.
 - *Logic Example:* When `generate_stripe_link` is called, it uses the `stripe` Python library to create a link, saves the link URL to the `Invoice` model, and returns it to the frontend.
- `urls.py`: Maps URLs (like `/finance/`) to specific functions in `views.py`.
- `forms.py`: Handles data validation for forms (e.g., ensuring a user enters a valid email or positive number).
- `admin.py`: Customizes the default Django Admin panel for superusers.

[core/templates/](#) (Frontend UI)

- Contains HTML files ([dashboard.html](#), [finance.html](#), [login.html](#)).
- Uses Django Template Language (e.g., `{% for order in orders %}`) to dynamically inject database data into the HTML.
- Uses Bootstrap 5 for grid layouts and custom CSS variables for the dark theme.

[core/static/](#) & [media/](#)

- [static/](#) : Holds CSS files, JavaScript, and images.
- [media/](#) : Holds dynamically generated files, like the PDF invoices created by WeasyPrint

6. Deep Dive: Complex Features & Logic

A. Stripe Payment Integration

We use Stripe because it is the industry standard for secure payments.

1. **Payment Links:** Instead of building a complex credit card form (which requires heavy PCI compliance), we use Stripe's hosted "Payment Links". Django sends the order amount to Stripe, and Stripe returns a secure URL.
2. **Webhooks (Automatic Updates):** Stripe is configured to send a background HTTP POST request (Webhook) to our Django server ([/stripe/webhook/](#)) when a payment succeeds. Our [stripe_webhook](#) function verifies the security signature and automatically updates the database to "Paid".
3. **The "Sync" Fallback:** Because university networks or free hosting tiers sometimes block incoming Webhooks, we built a manual "**Sync Payment**" button. This button actively queries the Stripe API to check the payment status, ensuring the system never gets stuck.

B. Automated Email Receipts (Gmail SMTP)

When a payment is confirmed (via Webhook, Sync, or Manual Entry), the [send_payment_receipt_email\(\)](#) function is triggered.

1. It grabs the client's email from the [Order](#) model.
2. It uses [render_to_string](#) to inject the invoice details into a beautiful HTML email template ([payment_receipt.html](#)).

3. It uses Django's `send_mail` function, which connects to Gmail's SMTP server (Port 587) using an App Password to deliver the email.

C. PDF Invoice Generation

We use the WeasyPrint library. When the admin clicks "Download PDF", Django takes a specific HTML template, injects the order data, and WeasyPrint converts that HTML/CSS into a high-quality, printable PDF document saved in the `media/` folder

7. Security Features

- **Authentication:** All dashboard pages are protected by Django's `@login_required` decorator.
 - **CSRF Protection:** All forms use `{% csrf_token %}` to prevent Cross-Site Request Forgery attacks.
 - **Stripe Signature Verification:** The webhook endpoint verifies the cryptographic signature sent by Stripe to ensure the payment notification is genuinely from Stripe and not a hacker.
 - **Environment Variables:** Sensitive keys (Stripe keys, Gmail passwords) are kept out of the public code using `.env` files (or hardcoded safely for local testing).
-